

Strumenti IPC – parte 2

Come abbiamo già accennato, altri strumenti IPC sono:

- Le **Code di Messaggi**; simili alle pipe, da cui però differiscono per 2 aspetti:
 - i confini dei messaggi sono delimitati, così che **i lettori e gli scrittori comunicano in unità di messaggi**, e non in stream di byte privi di delimitazioni interne.
 - ciascun messaggio contiene un membro `type` di tipo intero, **ed è possibile selezionare i messaggi per tipo**, piuttosto che leggerli nell'ordine in cui sono stati scritti.
- I **Semafori**; permettono a molteplici processi di sincronizzare le proprie azioni.
 - Un semaforo è un valore intero mantenuto dal kernel visibile a tutti i processi che hanno i permessi necessari.
 - Un processo indica ai propri pari che sta eseguendo una qualche azione facendo una modifica al valore del Semaforo.
- La **Memoria Condivisa**; consente a molteplici processi di condividere lo stesso segmento di memoria.
 - Poiché l'accesso allo spazio della memoria utente è un'operazione veloce, la memoria condivisa è uno dei più veloci strumenti per l'IPC: una volta che un processo ha aggiornato la memoria condivisa, la modifica è immediatamente visibile agli altri processi che condividono lo stesso segmento.

```
ipcs → elenca le risorse IPC definite
ipcs -l → elenco dei limiti definiti per le risorse IPC
ipcmk → crea una nuova risorsa IPC
ipcrm → elimina una risorsa IPC
```

Come vedremo di seguito questi 3 IPC hanno qualcosa in comune:

Creazione/Apertura di Oggetti IPC

Ciascun meccanismo IPC ha associato una syscall get:

- `msgget()` per la Coda di Messaggi
- `semget()` per i Semafori
- `shmget()` per la Memoria Condivisa

Data una `key` intera, la chiamata `get`:

- crea un nuovo oggetto IPC con la `key` indicata e restituisce un identificatore unico per quell'oggetto; oppure
- restituisce l'identificatore di un oggetto IPC esistente e avente quella `key`

```
id = msgget(key, IPC_CREAT | S_IRUSR | S_IWUSR);  
if (id == -1)  
    errExit("msgget");
```

Constant	Octal value	Permission bit
S_ISUID	04000	Set-user-ID
S_ISGID	02000	Set-group-ID
S_ISVTX	01000	Sticky
S_IRUSR	0400	User-read
S_IWUSR	0200	User-write
S_IXUSR	0100	User-execute
S_IRGRP	040	Group-read
S_IWGRP	020	Group-write
S_IXGRP	010	Group-execute
S_IROTH	04	Other-read
S_IWOTH	02	Other-write
S_IXOTH	01	Other-execute

Specifichiamo i `flag` di accesso al nuovo oggetto, utilizzando le costanti elencate qui di fianco.

Inoltre, è possibile utilizzare i `flag` per altri scopi:

- Ogni processo che desideri accedere allo stesso oggetto IPC esegue una chiamata `get`, specificando la stessa `key` per ottenere lo stesso identificatore per quell'oggetto.
 - Se non esiste un oggetto IPC corrispondente alla `key`, ed è stata specificata la costante `IPC_CREAT`, la `get` crea un nuovo oggetto IPC.
- Un processo può garantire di essere il creatore di un oggetto IPC specificando il flag `IPC_EXCL`:
 - Se il flag `IPC_EXCL` è specificato, e l'oggetto IPC corrispondente alla `key` esiste già, la `get` fallisce con l'errore `EEXIST`.

Cancellazione di Oggetti IPC

La system call `ctl` per ciascun meccanismo di IPC sull'oggetto (`msgctl()`; `semctl()`; `shmctl()`) **esegue un gran numero di operazioni di controllo** che variano a seconda dell'IPC...

Tuttavia, ci sono operazioni comuni a tutti.

Esempio: `IPC_RMID` è **utilizzata per cancellare un oggetto**.

```
if (shmctl(id, IPC_RMID, NULL) == -1)
    errExit("shmctl");
```

ATTENZIONE: Persistenza degli oggetti IPC

Gli oggetti IPC hanno una *kernel persistence*: **dopo essere stati creati, continuano ad esistere finché sono esplicitamente cancellati o il sistema viene spento.**

- Per le Code di Messaggi e i Semafori, la cancellazione degli oggetti IPC è immediata, e qualsiasi informazione contenuta all'interno dell'oggetto è distrutta, a prescindere dal fatto che qualche altro processo stia ancora utilizzando quell'oggetto.
- La cancellazione di oggetti legati alla Memoria Condivisa ha un diverso comportamento.
 - Seguendo la chiamata `shmctl(id, IPC_RMID, NULL)`, il segmento di memoria condivisa è rimosso solo dopo che tutti i processi che lo utilizzano effettuano il *detach* (disconnessione) (molto più simile alla situazione della cancellazione di un file).

Tale proprietà degli oggetti IPC fornisce alcuni **Vantaggi**: è possibile per un processo creare un oggetto, modificarne lo stato e uscire, lasciando che l'oggetto resti accessibile da altri processi iniziati successivamente.

Contro:

- esistono limiti di sistema sul massimo numero di oggetti IPC di ogni tipo...
 - il problema è che questi oggetti sono connectionless, cioè il kernel non tiene traccia di quali processi hanno un oggetto aperto.
 - Quando si cancella una coda di messaggi, un'applicazione con molti processi può non essere agevolmente in grado di determinare quale sarà l'ultimo processo a richiedere l'accesso all'oggetto e quindi quando l'oggetto può essere cancellato senza problemi.
-

IPC Keys

- Le `keys` dei meccanismi IPC sono valori interi rappresentati con il tipo `key_t`.
- La chiamata `get` mappa una `key` sul corrispondente identificatore IPC intero.
- Queste chiamate garantiscono che:
 - se creiamo un nuovo oggetto, quell'oggetto abbia un identificatore unico
 - se specifichiamo la `key` di un oggetto esistente, otteniamo sempre lo stesso identificatore per quell'oggetto.

Come si genera una `key` unica? Abbiamo 3 modi:

1. Scelta Casuale di un valore intero, che è tipicamente memorizzato in un header file incluso da tutti i programmi che usano l'oggetto IPC.
2. Utilizzo della costante `IPC_PRIVATE` come valore della `key` nella invocazione alla `get` al momento della creazione dell'oggetto, **che produce sempre un oggetto con una chiave unica.**

```
id = msgget(IPC_PRIVATE, S_IRUSR | S_IWUSR);
```

- In questo caso non è necessario specificare i flag `IPC_CREAT` o `IPC_EXCL`.
- Questa tecnica è particolarmente utile in applicazioni con molti processi in cui il processo padre crea l'oggetto IPC prima di eseguire la `fork()`, con il risultato che il figlio eredita l'identificatore dell'oggetto IPC.

3. Utilizzo della funzione `ftok()` per generare una `key` *molto probabilmente* unica.

```
#include <sys/ipc.h>
key_t ftok(char *pathname, int proj);
//Returns integer key on success, or -1 on error
```

- Questo valore di `key` è generato dal `pathname` fornito e dal valore `proj` utilizzando un algoritmo definito a livello di implementazione.
- Il fine del valore `proj` è consentire di generare diverse `key` a partire dallo stesso file; è utile quando un'applicazione deve creare vari oggetti IPC dello stesso tipo.
- Storicamente, l'argomento `proj` era di tipo `char`, ed è spesso specificato come tale nelle chiamate a `ftok()`.

```
key_t key;
int id;
key = ftok("/mydir/myfile", 'x');
...
id = msgget(key, IPC_CREAT | S_IRUSR | S_IWUSR);
...
```

Gestione dei Permessi su un oggetto IPC

Il kernel mantiene una struttura dati per ogni istanza di un oggetto IPC:

- La forma di questa struttura dati varia a seconda del meccanismo IPC ed è definito nell'header file corrispondente a ciascun meccanismo IPC
- La struttura dati associata a un oggetto IPC è **inizializzata quando l'oggetto è creato per mezzo dell'appropriata system call get**.
 - Una volta che l'oggetto è stato creato, un programma può ottenere una copia di questa struttura dati utilizzando la syscall `ctl`, e specificando un'operazione di tipo `IPC_STAT`.
- Alcuni elementi della struttura dati possono essere modificati per mezzo della operazione `IPC_SET`.
- Il campo `mode` della sottostruttura `ipc_perm` contiene i permessi per l'oggetto IPC.
 - I permessi sono inizializzati utilizzando i 9 bit più bassi specificati nei flag della syscall `get`, ma possono essere modificati successivamente utilizzando l'operazione `IPC_SET`
 - Come con i file, i permessi sono divisi in tre categorie: owner (o user), group, e other, ed è possibile specificare diversi permessi per ogni categoria.

```
struct ipc_perm {
    key_t    key;
    uid_t    uid; /* ushort: Owner's user ID */
    gid_t    gid; /* ushort: Owner's group ID */
    uid_t    cuid; /* Creator's user ID */
    gid_t    cgid; /* Creator's group ID */
    unsigned short mode; /* permissions */
    unsigned short __seq; /*Sequence number*/
};
```

```
struct shmid_ds shmds;

if (shmctl(id, IPC_STAT, &shmds) == -1) // prelevo dal kernel
    // ... gestione errore ...

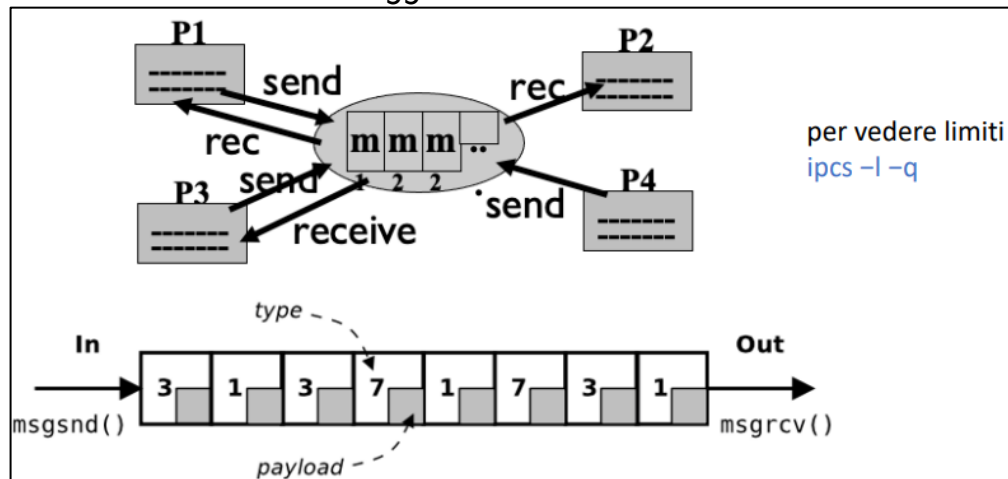
shmds.shm_perm.uid = newuid; // modifico l'owner ID
                             // (nella copia locale)

if (shmctl(id, IPC_SET, &shmds) == -1) // modifico la copia
    // del kernel
    // ... gestione errore ...
```

Esempio: Modifica del campo `uid` per un segmento di memoria condivisa.
La struttura dati associata è di tipo `shmid_ds`.

Code di Messaggi

Le **Code di Messaggi** sono un canale di comunicazione su cui possono affacciarsi più processi (anche non correlati) che inviano e ricevono messaggi:



Le Code di Messaggi differiscono da Pipe e FIFO per queste caratteristiche (già accennate):

- La loro creazione avviene con la chiamata a `msgget()`.
- La comunicazione per mezzo di code di messaggi è 'message-oriented'; cioè, **Il lettore riceve messaggi interi, scritti dallo scrittore**.
 - Non è possibile leggere porzioni di messaggi, lasciandone altre porzioni in coda, o leggere più messaggi alla volta.
 - Oltre a contenere dati, ogni messaggio contiene un membro di tipo intero che permette di prelevare i messaggi dalla coda in ordine (first-in, first-out) oppure per tipo.

Creazione/Apertura di una Coda di Messaggi

```
#include <sys/types.h> /* For portability */
#include <sys/msg.h>
int msgget(key_t key, int msgflg);
//Returns message queue identifier on success, or -1 on error
```

- L'argomento `key` è una chiave generata utilizzando un numero casuale, `IPC_PRIVATE` o `ftok()`;
- L'argomento `msgflg` è una maschera di bit che **specifica i permessi da associare a una nuova coda di messaggi**. Se la coda esiste già, permette di verificarne i permessi.
 - `IPC_CREAT`: se non esiste una coda con la `key` specificata, crea una nuova coda;
 - `IPC_EXCL`: se è presente anche `IPC_CREAT`, e una coda con la `key` specificata esiste già, restituisci un fallimento con errore `EEXIST`
- La system call `msgget()` inizia cercando all'interno delle code di messaggi esistenti quella con la `key` specificata.
 - **Se tale `key` corrisponde a una coda**,
 - la `msgget()` restituisce l'identificatore di quella coda (a meno che siano stati specificati sia `IPC_CREAT` sia `IPC_EXCL`, nel qual caso viene restituito un errore).
 - **Se la coda non esiste e `IPC_CREAT` è specificato**,
 - la `msgget()` crea una nuova coda e ne restituisce l'identificatore al chiamante.

Condivisione del Valore di Key

La condivisione del valore di key può avvenire in diversi modi:

1. Se la coda viene usata da processi in relazione padre-figlio si può sfruttare il fatto che un figlio eredita copia delle variabili da padre.
 - a. Se viene eseguita una `execve` basta passare la chiave come argomento della funzione
2. In un file di definizioni `f_header.h`, incluso da tutti i processi che devono usare la stessa coda:
`#define MYKEY 1234`
 - a. Il processo responsabile per l'allocazione della coda eseguirà:
 - i. `int q_id = msgget(MYKEY, IPC_CREAT | 0666);`
 - b. Un processo che deve usare la coda associata a MYKEY eseguirà:
 - i. `int q_id = msgget(MYKEY, 0);`
 - ii. Se la coda associata a MYKEY esiste, viene restituito il suo identificatore, altrimenti viene restituito `-1`

Invio/Ricezione di Messaggi

Le system call `msgsnd()` e `msgrcv()` eseguono le operazioni di I/O sulle code di messaggi.

- Il primo argomento in entrambe le chiamate (`msqid`) è un identificatore di coda di messaggi.
- Il secondo argomento, `msgp`, è un puntatore a una struttura 'programmerdefined' utilizzata per contenere il messaggio inviato o ricevuto.

Questa struttura ha la seguente forma:

```
struct mmsg {  
    long mtype;    /* Message type */  
    ... mtext ... /* Message body */  
}
```

Invio:

```
#include <sys/types.h> /* For portability */  
#include <sys/msg.h>  
int msgsnd( int msqid, const void *msgp,  
            size_t msgsz, int msgflg );  
//Returns 0 on success, or -1 on error
```

La system call `msgsnd()` scrive un messaggio su una coda di messaggi.

- Per inviare un messaggio con la `msgsnd()`, è necessario assegnare il membro `mtype` della struttura a un valore maggiore di 0 e copiare i dati da trasmettere nei membri della struttura.
 - L'argomento `msgsz` specifica il numero di bytes contenuti nel membro `mtext` della struttura
 - L'ultimo argomento, `msgflg`, è una maschera di bit dei flag che controllano l'operazione di `msgsnd()`: In questo specifico caso di IPC è definito un solo un flag:
 - `IPC_NOWAIT`. Consente di eseguire una 'nonblocking send'.
 - Di norma, se una coda è piena, `msgsnd()` si blocca finché non si libera abbastanza spazio per il messaggio che si desidera aggiungere.
 - Invece, se è specificato questo flag, la `msgsnd()` restituisce immediatamente con l'errore `EAGAIN`.
-

Ricezione:

```
#include <sys/types.h> /* For portability */
#include <sys/msg.h>
ssize_t msgrcv(int msqid, void *msgp,
               size_t maxmsgsz, long msgtyp, int msgflg);
//Returns number of bytes copied into mtext field, or -1 on
error
```

- maxmsgsz: specifica la capienza del membro mtext del buffer msgp
 - Se il corpo del messaggio da rimuovere dalla coda supera maxmsgsz bytes, nessun messaggio viene rimosso dalla coda, e la msgrcv() fallisce con errore E2BIG.
- Non necessariamente i messaggi vengono letti nell'ordine con cui sono stati scritti e inviati alla coda. È possibile selezionarli utilizzando il valore contenuto nel membro mtype. Questa selezione è controllata dall'argomento msgtyp:
 - se msgtyp è uguale a 0, viene prelevato il primo messaggio dalla coda e restituito al processo chiamante;
 - se msgtyp è maggiore di 0, viene prelevato il primo messaggio il cui mtype è uguale a msgtyp e restituito al chiamante.
 - specificando diversi valori per msgtyp, vari processi possono leggere da una coda di messaggi senza competere (racing) per leggere gli stessi messaggi. Una tecnica utile è quella in cui ciascun processo seleziona messaggi contenenti il proprio PID.
 - se msgtyp è minore di 0, la coda è trattata come una coda con priorità.
 - Viene prelevato e restituito per primo il messaggio con il minimo mtype minore o uguale al valore assoluto di msgtyp.

Esempio:

```
msgrcv(id, &msg, maxmsgsz, -300, 0);
```

Queste chiamate msgrcv() preleverebbero i messaggi nell'ordine:

- 2 (type 100),
- 5 (type 100),
- 3 (type 200), e
- 1 (type 300).
- Un'ulteriore chiamata si bloccherebbe poiché il type dell'ultimo messaggio (400) supera 300.

queue position	Message type (mtype)	Message body (mtext)
1	300	...
2	100	...
3	200	...
4	400	...
5	100	...

- L'argomento msgflg è una maschera di bit formata mettendo in OR zero o più flag:
 - IPC_NOWAIT Esegui una ricezione 'nonblocking'.
 - Normalmente, se sulla coda non sono presenti messaggi con il msgtyp specificato, msgrcv() si blocca fino a quando tale messaggio non diviene disponibile.
 - Specificando il flag IPC_NOWAIT comporta che in questo caso la msgrcv() ritorni immediatamente con errore ENOMSG.
 - MSG_NOERROR.
 - Di default, se la dimensione dell'mtext eccede lo spazio disponibile (definito dall'argomento maxmsgsz), msgrcv() fallisce.
 - Se viene specificato il flag MSG_NOERROR, la msgrcv() rimuove il messaggio dalla coda, ne tronca l'mtext a maxmsgsz bytes, e lo restituisce al chiamante.
 - I dati troncati sono persi

Invio & Ricezione: Errori Possibili

- In caso di errore, ritornano -1
 - Valori di errno
 - EACCES: no permission to operate
 - Eseguita `msgsnd()` senza averne i permessi
 - Eseguita `msgrcv()` senza averne i permessi
 - EIDRM: la coda di messaggi è stata rimossa
 - EINTR: the process caught a signal while sleeping
 - Quando bloccato sulla `msgsnd()` per coda piena
 - Quando bloccato su `msgrcv()` per messaggio richiesto non presente
 - ENOMEM, E2BIG: raggiunti i limiti di sistema
-

Operazioni di Controllo sulla Coda di Messaggi

```
#include <sys/types.h> /* For portability */
#include <sys/msg.h>

int msgctl(int msqid, int cmd, struct msqid_ds *buf);
//Returns 0 on success, or -1 on error
```

- L'argomento `cmd` specifica l'operazione da eseguire sulla coda:
 - `IPC_RMID`:
 - Rimuove immediatamente la coda di messaggi e la sua associata struttura dati `msqid_ds`. Tutti i messaggi presenti sulla coda vanno persi e qualsiasi processo Lettore o scrittore in attesa sulla coda è immediatamente svegliato, con la `msgsnd()` o la `msgrcv()` che falliscono con errore `EIDRM`.
 - Il terzo argomento della `msgctl()` è ignorato per questa operazione.
 - `IPC_STAT`; Copia la struttura `msqid_ds` nel buffer puntato da `buf`.
 - `IPC_SET`; Aggiorna i membri della struttura `msqid_ds` associata con questa coda di messaggi, utilizzando i valori presenti nel buffer puntato da `buf`.